

Skip Lists

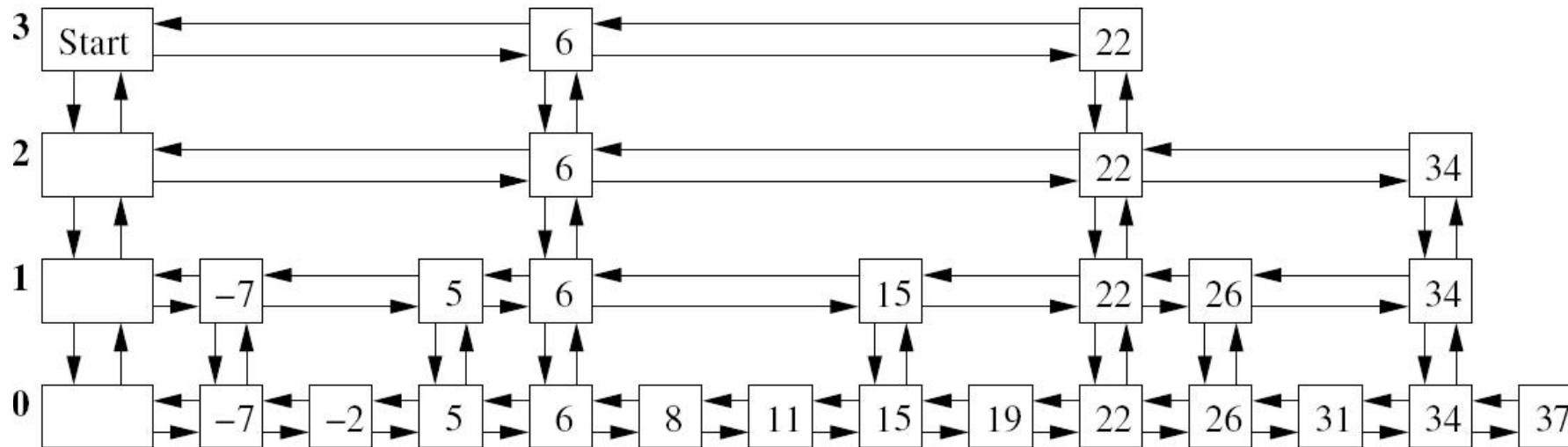
Design and Analysis of Algorithms

Brian C. Dean

**School of Computing
Clemson University**

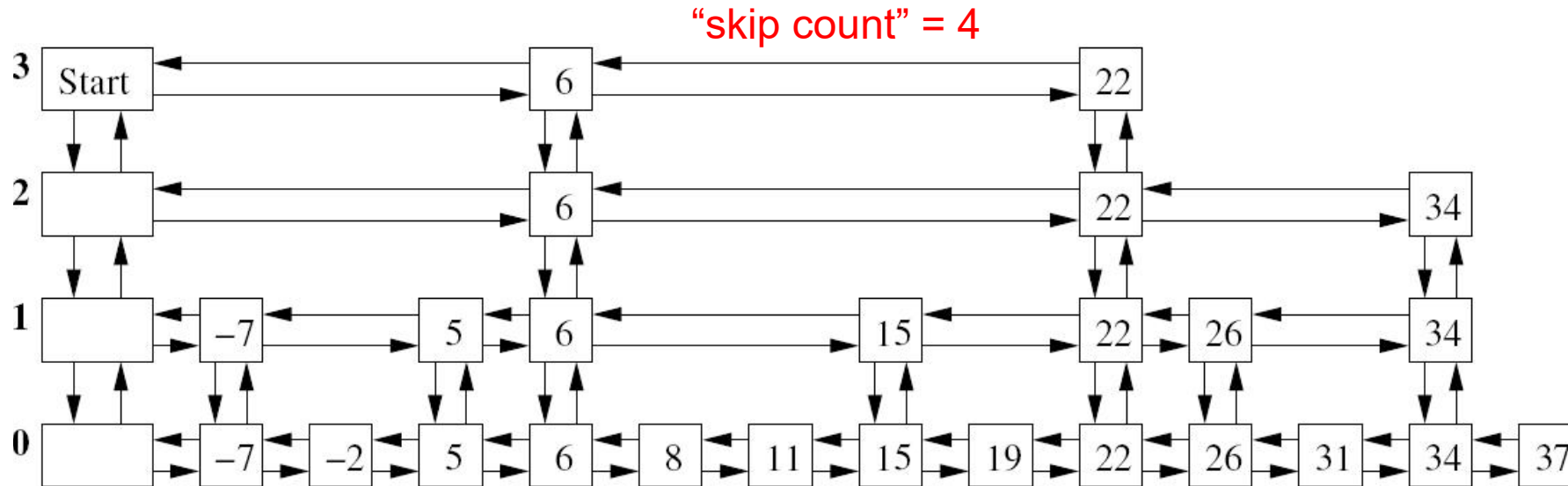


The Skip List : A Linked List With Added “Express Lanes”



- Can implement a set, map, or sequence with logarithmic (expected) performance on all major operations – same as a balanced BST.
- Each level contains roughly $\frac{1}{2}$ elements of level below.
- *find(x)*: move right unless that would overshoot, otherwise down.

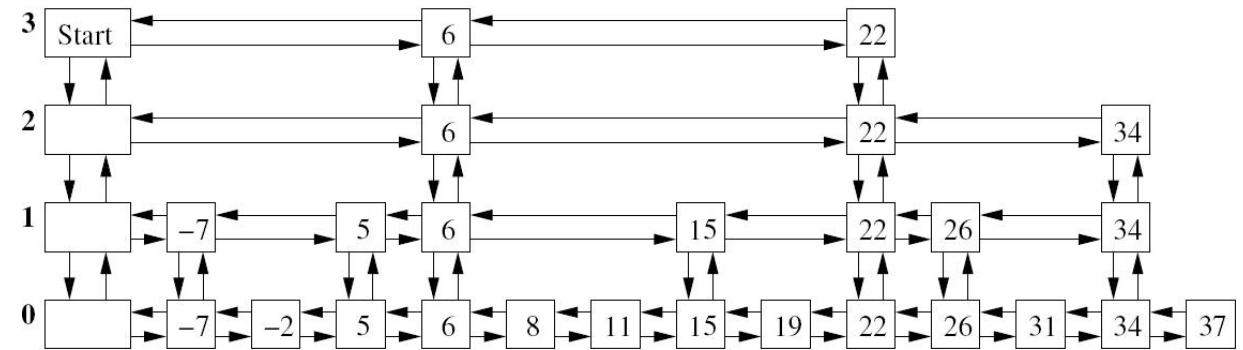
The Skip List : A Linked List With Added “Express Lanes”



- Can implement a set, map, or sequence with logarithmic (expected) performance on all major operations – same as a balanced BST.
- Supports *find* (value-based access) or *select* (rank-based access, used also for sequences). For select, augment with **skip counts**.

A Clever Use of Randomization...

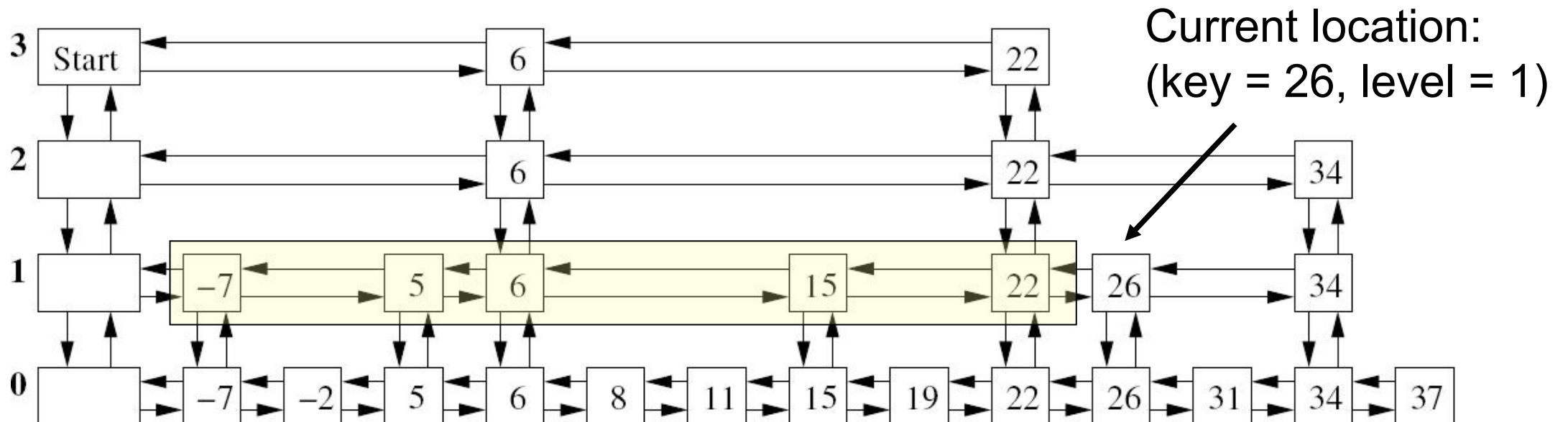
- To *insert* a new element,
 - Insert into the level-0 (bottom) list.
 - Flip a fair coin. If heads, also insert into the level-1 list, then flip another fair coin, and if heads again, insert into level-2 list, etc.



- This guarantees that:
 - By randomized reduction, $\mathbf{E}[\text{number of levels}] = O(\log n)$.
 - Also by randomized reduction, $\mathbf{E}[\text{running time of } find] = O(\log n)$.
(so the same expected running time applies to *insert*, *delete*, etc.)

Find Runs in $O(\log n)$ Expected Time

- Consider equivalent backward “unfind” process.
- Look at number of elements to our left in current level.
- With probability $\geq \frac{1}{4}$, this reduces to $\leq \frac{1}{2}$ its current value at each step of unfind $\rightarrow O(\log n)$ expected steps by randomized reduction.

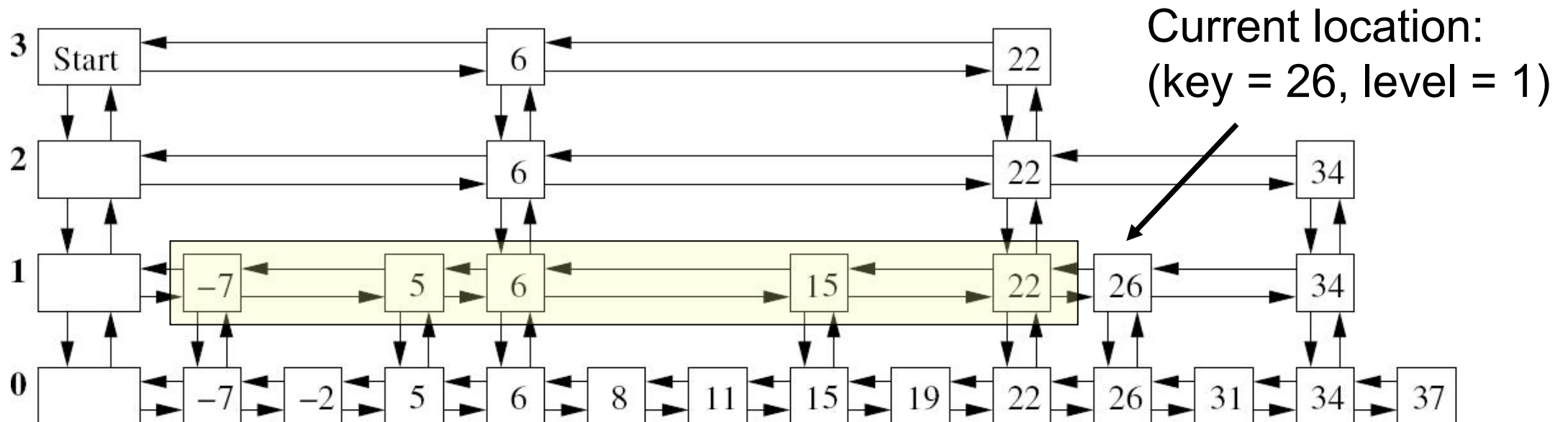


Find Runs in $O(\log n)$ Expected Time

$\Pr[\text{flipped heads at current location to move up}] = \frac{1}{2}$

If so, $\Pr[\leq \frac{1}{2} \text{ elements to left also moved up}] \geq \frac{1}{2}$

- With probability $\geq \frac{1}{4}$, this reduces to $\leq \frac{1}{2}$ its current value at each step of unfind $\rightarrow O(\log n)$ expected steps by randomized reduction.



Skip Lists Versus BSTs

- 1-1 mapping between skip lists and BSTs, leading to alternative ways to keep BSTs balanced.

Exploring the Duality Between Skip Lists and Binary Search Trees

Brian C. Dean
School of Computing
Clemson University
Clemson, SC
bcdean@cs.clemson.edu

Zachary H. Jones
School of Computing
Clemson University
Clemson, SC
zjones@cs.clemson.edu

ABSTRACT

Although skip lists were introduced as an alternative to balanced binary search trees (BSTs), we show that the skip list can be interpreted as a type of randomly-balanced BST whose simplicity and elegance is arguably on par with that of today's most popular BST balancing mechanisms. In this paper, we provide a clear, concise description and analysis of the "BST" interpretation of the skip list, and compare it to similar randomized BST balancing mechanisms. In addition, we show that any rotation-based BST balancing mechanism can be implemented in a simple fashion using a skip list.

1. INTRODUCTION

Since its introduction in 1989 [12, 13], the skip list has

statically optimal [5] skip lists have already been independently derived in the literature).

That the skip list can be interpreted as a type of randomly-balanced tree is not particularly surprising, and this has certainly not escaped the attention of other authors [2, 10, 9, 8]. However, essentially every tree interpretation of the skip list in the literature seems to focus entirely on casting the skip list as a randomly-balanced multiway branching tree (e.g., a randomized *B*-tree [4]). Messeguer [8] calls this structure the *skip tree*. Since there are several well-known ways to represent a multiway branching search tree as a BST (e.g., replace each multiway branching node with a miniature balanced BST, or replace (first child, next sibling) with (left child, right child) pointers), it is clear that the skip list can in principle also be represented by a BST. Munro et al. state precisely this in the concluding remarks of [10],

Applications Based on Common Ideas

- Navigating quickly in other data structure problems, e.g.,
 - Computing *level ancestors* or *lowest common ancestors* in trees.
 - Finding a target machine in a distributed hash table (next module).
- “Skip links” in deep neural net architectures to mitigate “vanishing gradients” in training.

